P3313R0
Impacts of noexept on ARM table based exception metadata

Draft Technical Report, 2024-05-22

This version:
https://wg21.link/P3313R0

Author:
Khalil Estell

Audience:
LEWG, LWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

C++ exceptions serve as the core mechanism for propagating errors from their detection points to respective handlers. The `noexcept` keyword informs the compiler that a function is guaranteed to not emit exceptions. This assurance is purported to enable compilers to undertake specific optimizations that can enhance code gen significantly. This paper examines the influence of the noexcept keyword on the management of exception metadata and cleanup landing pads, specifically within the framework of "zero-cost" table-based Itanium exception handling mechanisms utilized by the ARM Exception Handling ABI (ARM EHABI). This study shows that the current strategies for selecting exception metadata for functions by the GCC toolchain is often suboptimal. The propose improvements include better selection of exception metadata for functions, grouping functions with identical exception metadata so they can be merged, and link-time analysis to determine the implicit noexcept status of functions. This approach would provide the desired effect of using noexcept in as many places as possible without the cost of marking each function as such.

Table of Contents

§ 1. Objective

The primary goal of this study is to evaluate the impact of the noexcept keyword and the invocation of noexcept functions under various conditions. Specifically, the research aims to assess how noexcept influences an application's binary size through modifications in the exception index, exception table, and the code generation of the function.

The following questions will be addressed:

1. How does labeling a function as noexcept alter its metadata?
2. What changes occur in a function's metadata when it calls noexcept functions?
3. What implications arise when a try block exclusively calls noexcept functions?
4. How does the interaction with noexcept functions affect functions that manage objects with non-trivial destructors?

The research will conclude with recommendations for optimizing code generation.

§ 2. Background

This paper will focus on the Itanium table based exceptions on ARM as used by GCC. This choice comes from the author's experience with this form of exception handling and this architecture. The insights provided here should be consistent with

other forms of table based exception handling. All data related to the exception data structures can be found at this link: ARM-software/abi-aa: ehabi32.rst

The GCC LSDA is an exception to this which can be found in the document HP Exception Handling Tables aC++ A.01.15.

2.1. ARM Exception Index

In the ARM Exception Handling ABI, each function involved in exception unwinding is assigned a unique entry within the exception index. This entry comprises two 32-bit words:

- The first word represents a position-relative, 31-bit offset to the function's starting address.
- The second word contains either inline unwind instructions or a position-relative, 31-bit offset to the function's exception metadata.

```
struct arm_index_entry
{
 std::uint32_t function;
 std::uint32_t content;
};
```

The placement of each entry is relative to the position of its corrisponding function in the `.text` section of the program. For example if the `.text` section starts with function `foo`, then function `bar`, then function `baz`, then the exception index's first entries be for `foo`, `bar` and `baz` in that order. The index MUST be binary searchable using the program counter as the search term. The goal is to find the index entry associated with the function that the program counter is currently within the bounds of. This is checked by performing the following this expression:

```
void* entry_function_address = to_absolute_address(&entry[i].function);
void* next_entry_function_address = to_absolute_address(&entry[i + 1].function);

if (entry_function_address <= PC && PC < next_entry_function_address) {
    return entry[i];
}
```

The interpretation of the content field is determined by the status of the 31st bit:

- If the 31st bit is set to `1`, the content field holds inline unwind instructions, accommodating up to three bytes of such instructions.
- If the content is exactly `0x1`, it signifies the `CANNOT_UNWIND` flag, instructing the exception runtime that unwinding cannot proceed for this function. The runtime will terminate.
- If the 31st bit is `0`, the remaining bits represent a `prel31` offset, a 31-bit position-relative offset.

To compute the absolute address from a `prel31` offset, perform the following steps:

1. Sign-extend the value from 31 bits to a full 32-bit integer.
2. Convert the result into a `int32_t` to obtain the correct 32-bit signed offset.
3. Add this offset to the base address of the content field within the exception index.

This process yields the absolute address where the function's exception metadata is located.

§ 2.2. ARM Unwind Instructions

ARM EHABI unwind instructions are compactly encoded in single-byte increments. In contrast, ARM THUMB2 instructions, utilized by the Cortex M series of microcontrollers, range from 2 to 4 bytes in size, with a typical ARM instruction occupying 4 bytes. Consequently, unwind instructions are significantly smaller, ranging from half to a quarter the size of standard instructions. This compact size is sufficient for executing all necessary tasks involved in unwinding a frame, which include:

1. Deallocating local variables by adjusting the stack pointer.
2. Restoring general-purpose registers from the stack to the CPU.
3. Transferring special-purpose register contents from the stack to the appropriate coprocessor.

Below are some of the common unwind instructions:

ARM Unwind Instructions

Instruction (binary)	Explanation
00xxxxxx	vsp = vsp + (xxxxxx < < 2) + 4. Covers range 0x04 - 0x100 inclusive
10000000 00000000	Refuse to unwind (for example, out of a cleanup)
10100nnn	Pop r4-r[4+nnn]
10101nnn	Pop r4-r[4+nnn], r14
10110001 0000iiii	Pop integer registers under mask {r3, r2, r1, r0}
10110000	Finish

§ 2.3. ARM Personality Routine

There are three forms of ARM personality:

- **SU16**: short unwind with 16-bit descriptor scope.
- **LU16**: long unwind with 16-bit descriptor scope.
- **LU32**: long unwind with 32-bit descriptor scope.

Personality data descriptors detail regions within a function and the actions to take if the program counter is within one of these regions.

This document does not cover the ARM specific cleanup and catch descriptors as GCC typically employs a generic, cross-platform, and compressed representation of this data known as the GCC C++ LSDA (Language Specific Data Area).

No ARM personality routines can exceed 7 bytes of unwind instructions, as detailed in Appendix C of ehabi32.rst.

SU16 Layout:

- `[31]`: Personality indicator. Set to 1 if the data is a personality. Set to 0 if the content is a prel31 offset to additional data.
- `[30:28]`: Reserved.
- `[27:24]`: Personality index. For SU16, this is 0.
- `[23:16]`: Unwind instruction 1.
- `[15:8]`: Unwind instruction 2.
- `[7:0]`: Unwind instruction 3.

LU16 & LU32 Layout:

- **First word**:

- `[31]`: Personality indicator. Set to 1 if the data is a personality. Set to 0 if the content is a prel31 offset to additional data.
- `[30:28]`: Reserved.
- `[27:24]`: Personality index, can be 0, 1, or 2.
- `[23:16]`: Number of words following this one. Valid values are 1 or 2.
- `[15:8]`: Unwind instruction 1.
- `[7:0]`: Unwind instruction 2.

- **Second word**:
  - `[31:24]`: Unwind instruction 3.
  - `[23:16]`: Unwind instruction 4.
  - `[15:8]`: Unwind instruction 5.
  - `[7:0]`: Unwind instruction 6.
- **Third word (if applicable)**:
  - `[31:24]`: Unwind instruction 7.
  - `[23:16]`: `0xB0` (finish).
  - `[15:8]`: `0xB0` (finish).
  - `[7:0]`: `0xB0` (finish).

§ 2.4. GCC Language Specific Data Area (LSDA)

The ARM Exception Handling ABI (EHABI) for exception handling allows the integration of non-personality data as exception data, facilitating support for language-specific exception handling mechanisms. The EHABI permits the inclusion of custom language-specific data areas within the exception tables, a feature extensively utilized by GCC to substitute the default architecture-specific descriptors with its own. This functionality derives from a feature in the Itanium ABI, enabling various languages to implement their own functions for unwinding specific call frames. Consequently, not only can C++ exceptions be managed using GCC's language-specific data area, but other languages such as Java can also employ this area to control their exceptions.

For GCC's LSDA (Language Specific Data Area) format:

1. **Personality Function:** 32-bit value with the MSB set to 0. It contains a `prel31` offset to the function's handler, typically `__gxx_personality_v0` in GCC.
2. **Personality Data:** Architecture-specific unwind instructions.
3. **Header:** A variable-length sequence of bytes that delineates where DWARF information is located, the end of the type table, and the extent of the call site region. Entries in the DWARF location and type table can be marked with an omit flag `0xFF` to indicate their absence.
4. **Call Site Table:** Details the regions of the function associated with try scopes and cleanup. This table specifies the areas of the function that have particular actions assigned, as well as the location of the landing pad if an action is taken.
5. **Action Table:** Lists the indices to the types that can be caught for each call site region and specifies whether cleanup is required.
6. **Type Table:** Contains a unique set of `std::type_info` addresses for the types that can be caught within the function.

To gain a deeper understanding of how these regions are structured, refer to: HP Exception Handling Tables aC++ A.01.15.

§ 2.5. Destructor landing pads

```
08001140 <dtor::except_calls_all_except()>:
 8001140:  b500        push  {lr}
 8001142:  b085        sub  sp, #20
 8001144:  f7ff ff76   bl  8001034 <dtor::non_trivial_dtor::action() [clone .constprop.0]>
 8001148:  f7ff ff74   bl  8001034 <dtor::non_trivial_dtor::action() [clone .constprop.0]>
 800114c:  f7ff ff72   bl  8001034 <dtor::non_trivial_dtor::action() [clone .constprop.0]>
 8001150:  f7ff ff70   bl  8001034 <dtor::non_trivial_dtor::action() [clone .constprop.0]>
 8001154:  f7ff ff6e   bl  8001034 <dtor::non_trivial_dtor::action() [clone .constprop.0]>
 8001158:  f7ff ff6c   bl  8001034 <dtor::non_trivial_dtor::action() [clone .constprop.0]>
 800115c:  a803        add  r0, sp, #12
 800115e:  f7ff ff99   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
 8001162:  a802        add  r0, sp, #8
 8001164:  f7ff ff96   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
 8001168:  a801        add  r0, sp, #4
 800116a:  b005        add  sp, #20
 800116c:  f85d eb04   ldr.w  lr, [sp], #4
 8001170:  f7ff bf90   b.w  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
 8001174:  e005        b.n  8001182 <dtor::except_calls_all_except()+0x42>
 8001176:  a803        add  r0, sp, #12
 8001178:  f7ff ff8c   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
 800117c:  a802        add  r0, sp, #8
 800117e:  f7ff ff89   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
 8001182:  a801        add  r0, sp, #4
 8001184:  f7ff ff86   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
 8001188:  f000 fb26   bl  80017d8 <__cxa_end_cleanup>
 800118c:  e7f6        b.n  800117c <dtor::except_calls_all_except()+0x3c>
 800118e:  bf00        nop
```

Figure 1 *Full Function with Destructor Cleanup Region*

```
8001176:  a803        add  r0, sp, #12
8001178:  f7ff ff8c   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
800117c:  a802        add  r0, sp, #8
800117e:  f7ff ff89   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
8001182:  a801        add  r0, sp, #4
8001184:  f7ff ff86   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
8001188:  f000 fb26   bl  80017d8 <__cxa_end_cleanup>
```

Figure 2 *Isolated Destructor Cleanup Region*

Within this function, there are designated regions that the exception runtime targets to execute destructors for the current frame. The figures above depict ARM Thumb2 instructions. These instructions specifically load the address of an object from the stack into `R0`, which acts as the register for the first parameter in a function call, then the object's destructor is invoked. This process is the reverse of the construction sequence of the objects. The Itanium API utilized to re-enter the exception unwind flow after all necessary destructors are called is `__cxa_end_cleanup()`. The point at which the program counter re-enters the function is influenced by the scope where the exception propagation originated.

§ 2.6. Catch landing pads

```
80004fa: 2901        cmp r1, #1
80004fc: d001        beq.n 8000502 <main+0x18>
80004fe: f001 f98f   bl  8001820 <__cxa_end_cleanup>
8000502: f001 fa51   bl  80019a8 <__cxa_begin_catch>
8000506: 4a8f        ldr r2, [pc, #572]  @ (8000744 <main+0x25a>)
8000508: 6c53        ldr r3, [r2, #68] @ 0x44
800050a: 3301        adds  r3, #1
800050c: 6453        str r3, [r2, #68] @ 0x44
800050e: f001 fa8b   bl  8001a28 <__cxa_end_catch>
```

Figure 3 *Catch landing pad example*

According to Itanium, catch chains should be transformed into switch-case-like blocks. The unwinder sets `R0` with the exception object and `R1` with the case number for the corresponding block. The initial instruction in the assembly above, comparing `R1` with the number 1, determines the path of execution. If the comparison fails, the sequence proceeds to execute `__cxa_end_cleanup`, continuing the exception propagation. If the comparison is successful, the flow transitions to `__cxa_begin_catch`, executes the catch block, and concludes with `__cxa_end_catch`.

§ 3. Methods

To fulfill the objectives of this paper, a C++ application will be designed with a set of functions featuring noexcept and "except" functions in different usages. "except" in this case as a short hand for non-noexcept function. This application will self-assess during runtime by examining its exception table entries and the exception table. The investigation aims to provide insights into:

- Function exception rank
- LSDA size and the sizes of its sections (when applicable)

§ 3.1. Function Exception Rank

"function exception rank" refers to a classification scheme for functions based on their exception metadata's memory demands. This ranking system identifies:

1. **No index entry**: Indicates absence of an index table entry for functions where the compiler ascertains no exception propagation, utilizing zero memory.
2. **Inlined index data:** Involves direct inlining of unwind information into the index entry, negating the need for additional exception table space. This configuration employs an SU16 personality, consistently occupying 8 bytes due to ARM exception index ABI specifications.
3. **Table unwind instructions:** Applies when unwind details cannot be condensed into the 4-byte content section of the index, requiring 16 to 20 bytes—8 bytes for the index and 8 to 12 bytes for unwind instructions.
4. **GCC LSDA:** Positioned in the exception table, this data structure, although memory-intensive, effectively manages try/catch blocks and cleanup areas, starting at 28 bytes and increasing based on complexity.

§ 3.2. Experimental Setup

- **Toolchain**: Arm Gnu Toolchain 12.3

- **Instruction Set**: ARM THUMB 2
- **Target Processor**: Cortex M3 (executed on an stm32f103c8 microcontroller)
- **libc**: picolibc (to re-enable exception handling in the compiler)
- **Project URL**: https://github.com/kammce/cpp-papers/tree/main/noexcept
- **Debugging Technology**: `PyOCD` + `ST-Link V2`

All C++ functions in or used by an exhibit in the results section will be marked as `[[gnu::noinline]]` in order to prevent the compiler from inlining the functions.

The `/noexcept` directory in the repo provides a README file explaining how to build and execute the code.

§ 4. Results

All functions with the prefix "noexcept" are noexcept functions.

There exists an array called side_effect which is a `std::array` of `volatile std::uint32_t` numbers. This is to prevent the compiler from deducing the results of functions and garbage collecting most of the code.

Any exhibits with multiple rankings has been found to change their ranking depending on the position of the function in the code. For example, if the compiler sees that function A is inlined noexcept and the next function in the code (or symbol table) is function B and it has the same inline noexcept entry, the compiler will merge the two enteries and make a single entry for function B's. Because the next entry will be the next function with differing exception content, the when the binary search is performed to find the required entry, anything between function B and the next entry will have the same information. During unwinding, the selected entry will be function A if function B ever has an exception propagation reaches it. The behavior is the same, regardless. There maybe exhibits where this behavior also exists but was not tested in this paper.

§ 4.1. Exhibit 1: Leaf Function

Definition of `my_struct_t`:

```
struct my_struct_t
{
  int a;
  int b;
  int c;
};
```

```
void noexcept_initialize(my_struct_t& my_struct) noexcept
{
    my_struct.a = 17;
    my_struct.b = 22;
    my_struct.c = 33;
}
```

Rank 1: no entry
Rank 2: inlined noexcept

```
void initialize(my_struct_t& my_stru
{
    my_struct.a = 5;
    my_struct.b = 15;
    my_struct.c = 15;
}
```

Rank 1: no entry
Rank 2: inlined noexcept

Figure 4 *Leaf Functions*

§ 4.2. Exhibit 2: Calling only noexcept functions

```
void
noexcept_calls_all_noexcept() noexcept
{
    noexcept_bar();
    noexcept_baz();
    noexcept_qaz();
}
```

```
void
except_calls_all_noexcept()
{
    noexcept_bar();
    noexcept_baz();
    noexcept_qaz();
}
```

Rank 2: inlined noexcept	Rank 1: No entry Rank 2: inlined noexcept

Figure 5 *Calling all noexcept functions*

§ 4.3. Exhibit 3: Calling both except and noexcept functions

```
void
noexcept_calls_mixed() noexcept
{
    noexcept_bar();
    baz();
    noexcept_qaz();
}
```

```
void
except_calls_mixed()
{
    noexcept_bar();
    baz();
    noexcept_qaz();
}
```

Rank 4: GCC LSDA	Rank 3: Table Personality

Figure 6 *Calling a mix of except & noexcept functions*

§ 4.4. Exhibit 4: Functions calling only except function

```
void
noexcept_calls_all_except() noexcept
{
    bar();
    baz();
    qaz();
}
```

```
void
except_calls_all_except()
{
    bar();
    baz();
    qaz();
}
```

Rank 4: GCC LSDA	Rank 3: Table Personality

Figure 7 *Calling only except functions*

§ 4.5. Exhibit 5: Calling only noexcept in try/catch block

```
void
noexcept_calls_all_noexcept_in_try_catch() noexcept
{
    try {
        noexcept_bar();
        noexcept_baz();
    } catch (...) {
        side_effect[9] = side_effect[9] + 1;
    }
}
```

Rank 2: Inlined noexcept

```
void
except_calls_all_noexcept_in_try_catch()
{
    try {
        noexcept_bar();
        noexcept_baz();
    } catch (...) {
        side_effect[9] = side_effect[9] +
    }
}
```

Rank 1: No entry

Figure 8 *Calling only noexcept in try scope*

§ 4.6. Exhibit 6: Calling both except & noexcept in try/catch block

```
void
noexcept_calls_mixed_in_try_catch() noexcept
{
    try {
        bar();
        noexcept_baz();
    } catch (...) {
        side_effect[15] = side_effect[15] + 1;
    }
}
```

Rank 4: GCC LSDA

```
void
except_calling_mixed_in_try_catch()
{
    try {
        bar();
        noexcept_baz();
    } catch (...) {
        side_effect[22] = side_effect[22] + 1;
    }
}
```

Rank 4: GCC LSDA

Figure 9 *Calling mixed function types in a try scope*

§ 4.7. Exhibit 7: Calling only except in try/catch block

```
void
noexcept_calls_except_in_try_catch() noexcept
{
    try {
        bar();
        baz();
    } catch (...) {
        side_effect[17] = side_effect[17] + 1;
    }
}
```

Rank 4: GCC LSDA

```
void
except_calls_except_in_try_catch()
{
    try {
        bar();
        baz();
    } catch (...) {
        side_effect[8] = side_effect[8] + 1;
    }
}
```

Rank 4: GCC LSDA

Figure 10 *Calling only except functions in a try scope*

§ 4.8. Exhibit 8: Leaf class function

```
my_class::state_t
my_class::noexcept_state() noexcept
{
    return m_state;
}
```

Rank 1: No entry

```
my_class::state_t
my_class::state()
{
    return m_state;
}
```

Rank 1: No entry

Figure 11 *Typical function getter*

§ 4.9. Exhibit 9: Calling only noexcept with non-trivially destructable objects present

```
namespace dtor {
void
noexcept_calls_all_noexcept() noexcept
{
    non_trivial_dtor obj1;
    obj1.noexcept_action();
    non_trivial_dtor obj2;
    obj1.noexcept_action();
    obj2.noexcept_action();
    non_trivial_dtor obj3;
    obj1.noexcept_action();
    obj2.noexcept_action();
    obj3.noexcept_action();
}
}
```

Rank 1: No Entry

```
namespace dtor {
void
except_calls_all_noexcept()
{
    non_trivial_dtor obj1;
    obj1.noexcept_action();
    non_trivial_dtor obj2;
    obj1.noexcept_action();
    obj2.noexcept_action();
    non_trivial_dtor obj3;
    obj1.noexcept_action();
    obj2.noexcept_action();
    obj3.noexcept_action();
}
}
```

Rank 1: No Entry

Figure 12 *Calling only noexcept functions with non-trivially destructable objects present.*

§ 4.10. Exhibit 10: Calling only except with non-trivially destructable objects present

```
namespace dtor {
void
noexcept_calls_all_except() noexcept
{
    non_trivial_dtor obj1;
    obj1.action();
    non_trivial_dtor obj2;
    obj1.action();
    obj2.action();
    non_trivial_dtor obj3;
    obj1.action();
    obj2.action();
    obj3.action();
}
}
```

Rank 4: GCC LSDA

```
namespace dtor {
void
except_calls_all_except()
{
    non_trivial_dtor obj1;
    obj1.action();
    non_trivial_dtor obj2;
    obj1.action();
    obj2.action();
    non_trivial_dtor obj3;
    obj1.action();
    obj2.action();
    obj3.action();
}
}
```

Rank 4: GCC LSDA

Figure 13 *Calling only except with non-trivially destructable objects present*

§ 4.11. Exhibit 11

In this experiment, the function that is noexcept is moved down for each of the following functions. So in experiment 2, the first class function call to obj1 will become `noexcept_action()` and the second call after constructing obj2 will be `action()`. All other calls will be `noexcept_action()`.

```
namespace dtor {
void
noexcept_calls_experiment1() noexcept
{
    non_trivial_dtor obj1;
    obj1.action(); // experiment 1: calls action()
    non_trivial_dtor obj2;
    obj1.noexcept_action(); // experiment 2: calls action()
    obj2.noexcept_action(); // experiment 3: calls action()
    non_trivial_dtor obj3;
    obj1.noexcept_action(); // experiment 4: calls action()
    obj2.noexcept_action(); // experiment 5: calls action()
    obj3.noexcept_action(); // experiment 6: calls action()
}
}
```

- Noexcept:
  - Experiment 1-7: Rank 4 GCC LSDA
- Except:
  - Experiment 1-7: Rank 4 GCC LSDA

§ 4.12. LSDA Data

The data below is sorted by total size. The total size only accounts for the size of the memory in the LSDA region. It does not include the exception index entry nor the cleanup region in the function.

Function Name	Total Size	Max Action Offset	Type Table Offset	Call Site count	Call Site size	Action Table count	Action Table size	Type Table count	Type Table size
except_calling_mixed_in_try_catch	34	1	17	2	8	3	6	1	4
except_calls_except_in_try_catch	34	1	17	2	8	3	6	1	4
dtor::except_calls_all_except	32	0	0	4	16	0	0	0	0
dtor::except_calls_experiment7	32	0	0	4	16	0	0	0	0
noexcept_calls_mixed_in_try_catch	30	1	13	1	4	3	6	1	4
noexcept_calls_except_in_try_catch	30	1	13	1	4	3	6	1	4
dtor::except_calls_experiment1	24	0	0	2	8	0	0	0	0
dtor::except_calls_experiment2	24	0	0	2	8	0	0	0	0
dtor::except_calls_experiment3	24	0	0	2	8	0	0	0	0
dtor::except_calls_experiment4	24	0	0	2	8	0	0	0	0
dtor::except_calls_experiment5	24	0	0	2	8	0	0	0	0
dtor::except_calls_experiment6	24	0	0	2	8	0	0	0	0
noexcept_calls_mixed	16	0	0	0	0	0	0	0	0
noexcept_calls_all_except	16	0	0	0	0	0	0	0	0
dtor::noexcept_calls_all_except	16	0	0	0	0	0	0	0	0
dtor::noexcept_calls_experiment1	16	0	0	0	0	0	0	0	0
dtor::noexcept_calls_experiment2	16	0	0	0	0	0	0	0	0
dtor::noexcept_calls_experiment3	16	0	0	0	0	0	0	0	0
dtor::noexcept_calls_experiment4	16	0	0	0	0	0	0	0	0
dtor::noexcept_calls_experiment5	16	0	0	0	0	0	0	0	0
dtor::noexcept_calls_experiment6	16	0	0	0	0	0	0	0	0
dtor::noexcept_calls_experiment7	16	0	0	0	0	0	0	0	0

§ 4.13. Cleanup Landing Pads

> None of the `noexcept` functions had cleanup landing pads.

```
08001140 <dtor::except_calls_all_except()>:
# ...
8001176: a803        add r0, sp, #12
8001178: f7ff ff8c   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
800117c: a802        add r0, sp, #8
800117e: f7ff ff89   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
8001182: a801        add r0, sp, #4
8001184: f7ff ff86   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
8001188: f000 fb26   bl  80017d8 <__cxa_end_cleanup>
```

```
080012fc <dtor::except_calls_experiment1()>:
# ...
8001330: a801        add r0, sp, #4
8001332: f7ff feaf   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
```

```
8001336: f000 fa4f   bl  80017d8 <__cxa_end_cleanup>
```

```
0800133c <dtor::except_calls_experiment2()>:
# ...
8001370: a802        add r0, sp, #8
8001372: f7ff fe8f   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
8001376: a801        add r0, sp, #4
8001378: f7ff fe8c   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
800137c: f000 fa2c   bl  80017d8 <__cxa_end_cleanup>
```

```
08001380 <dtor::except_calls_experiment3()>:
# ...
80013b4: a802        add r0, sp, #8
80013b6: f7ff fe6d   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
80013ba: a801        add r0, sp, #4
80013bc: f7ff fe6a   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
80013c0: f000 fa0a   bl  80017d8 <__cxa_end_cleanup>
```

```
080013c4 <dtor::except_calls_experiment4()>:
# ...
80013f8: a803        add r0, sp, #12
80013fa: f7ff fe4b   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
80013fe: a802        add r0, sp, #8
8001400: f7ff fe48   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
8001404: a801        add r0, sp, #4
8001406: f7ff fe45   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
800140a: f000 f9e5   bl  80017d8 <__cxa_end_cleanup>
```

```
08001410 <dtor::except_calls_experiment5()>:
# ...
8001444: a803        add r0, sp, #12
8001446: f7ff fe25   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
800144a: a802        add r0, sp, #8
800144c: f7ff fe22   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
8001450: a801        add r0, sp, #4
8001452: f7ff fe1f   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
8001456: f000 f9bf   bl  80017d8 <__cxa_end_cleanup>
```

```
0800145c <dtor::except_calls_experiment6()>:
# ...
8001490: a803        add r0, sp, #12
8001492: f7ff fdff   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
8001496: a802        add r0, sp, #8
8001498: f7ff fdfc   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
800149c: a801        add r0, sp, #4
800149e: f7ff fdf9   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
80014a2: f000 f999   bl  80017d8 <__cxa_end_cleanup>
```

```
080014a8 <dtor::except_calls_experiment7()>:
# ...
80014de: a803        add r0, sp, #12
80014e0: f7ff fdd8   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
80014e4: a802        add r0, sp, #8
```

```
80014e6: f7ff fdd5   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
80014ea: a801        add r0, sp, #4
80014ec: f7ff fdd2   bl  8001094 <dtor::non_trivial_dtor::~non_trivial_dtor()>
80014f0: f000 f972   bl  80017d8 <__cxa_end_cleanup>
```

The cleanup regions follow a very consistent pattern. Set `R0` to the address of the object to be destroyed and call the destructor. It takes 2 bytes to load `R0`, 4 bytes to call a destructor, and 4 bytes to call `__cxa_end_cleanup`. So the total cost in bytes for the number of objects that must be destroyed in a frame is `size(n) = 4 + 6n`.

5. Analysis

5.1. Leaf Functions

Consider § 4.1 Exhibit 1: Leaf Function and § 4.8 Exhibit 8: Leaf class function. Both exhibits contain leaf functions. § 4.8 Exhibit 8: Leaf class function has no exception index entry. This makes sense since the function does not throw an exception and does not call any other functions. There is no possibility that an exception will ever propagate from such a function. Thus their index entries can be omitted from the table.

If the functions of § 4.1 Exhibit 1: Leaf Function are moved between other functions that have exception index entries, then they will both get exception index entries with an inlined noexcept marker. GCC is able to deduce that the `initialize` functions never calls other functions and thus is de facto noexcept. But rather than eliminate the entry, the compiler decides to provide an entry for both.

These entries do not need to exist and being able to omit them would free up 8 bytes of space per function. This seems like an opportunity for the compiler to be improved.

5.2. Calling only noexcept functions

§ 4.2 Exhibit 2: Calling only noexcept functions contains functions calling only noexcept functions. The noexcept function in this exhibit gets a rank 4 whereas the except function just gets a rank 3. In both cases, the compiler should have opted to omit the exception data entirely. These functions are never reachable via exception propagation.

The compiler has chosen data structures for these two functions way above what is necessary for them.

5.3. Compiler Making Bad Choices

§ 4.3 Exhibit 3: Calling both except and noexcept functions and § 4.4 Exhibit 4: Functions calling only except function match each other in their rankings. Both either use an except function or a mix of except and noexcept functions. None of the functions require cleanup or have any catch blocks.

Yet, GCC chooses a mostly empty LSDA data structure for the noexcept functions. An inline noexcept flag would have worked as there is no objects to cleanup or catch blocks to consider.

The except functions have enough unwind instructions to warrant their data being placed in the exception table. Meaning these very simple functions were not unwindable in a 3 unwind instructions.

```
080015a8 <except_calls_all_except()>:
80015a8: b508        push  {r3, lr}
80015aa: f7ff fcdd   bl  8000f68 <bar()>
80015ae: f7ff fcfb   bl  8000fa8 <baz()>
```

```
80015b2: e8bd 4008   ldmia.w sp!, {r3, lr}
80015b6: f7ff bd17   b.w 8000fe8 <qaz()>
80015ba: bf00        nop
```

This function disassembly does two strange things. It pushes `R3` onto the stack which requires 2 bytes of unwind information to be unwound. It also performs some stack manipulation before calling the last function. If the compiler had chosen to use `R4` rather than `R3` and called the `qaz()` function normally, then only a single byte of instruction memory would be required to unwind it, specifically `0b10101000` (`0xA8`). This optimization could bring the exception rank to rank 2.

All of these functions should have rank 2, inlined noexcept flag and inline personality.

§ 5.4. Except Poisoning

A consistent observation across all exhibits is that calling a single except function, elevates the function's rank to at least rank 2, accompanied by the necessary cleanup landing pads.

Thus, the advantages of calling `noexcept` functions are negated by the introduction of any except function.

§ 5.5. Reducing Cleanup Landing Pads

Marking a C++ function as `noexcept` completely eliminates the cleanup regions. Such regions become unreachable, making their inclusion unnecessary.

However, in an except functions, if the only functions called after the construction of an object are `noexcept` functions, then the destructor call for that object can be omitted from the cleanup landing area. Calling a single except function afterwards will result in the object's destructor call being added to the cleanup landing pad.

§ 5.6. Eliminating Try/Catch Blocks

§ 4.5 Exhibit 5: Calling only noexcept in try/catch block presents an interesting scenario where the compiler assigns a rank of 2 to the `noexcept` function and a rank of 1 to the except function. The rank of 1 may be influenced by the placement of the functions within the source code. Nevertheless, the compiler opted for suitably minimal options for the code. The disassembly for both is minimal and excludes any record of the catch blocks. The compiler successfully determined that the catch blocks were unreachable and omitted them.

```
080015bc <noexcept_calls_all_noexcept_in_try_catch()>:
80015bc: b508        push  {r3, lr}
80015be: f7ff fc9d   bl  8000efc <noexcept_bar()>
80015c2: e8bd 4008   ldmia.w sp!, {r3, lr}
80015c6: f7ff bcab   b.w 8000f20 <noexcept_baz()>
80015ca: bf00        nop
```

```
080015cc <except_calls_all_noexcept_in_try_catch()>:
80015cc: b508        push  {r3, lr}
80015ce: f7ff fc95   bl  8000efc <noexcept_bar()>
80015d2: e8bd 4008   ldmia.w sp!, {r3, lr}
80015d6: f7ff bca3   b.w 8000f20 <noexcept_baz()>
80015da: bf00        nop
```

Note that calling all noexcept functions within a try block is a code smell. It begs the question of, "what exception were you expecting to catch from these APIs?"

§ 4.6 Exhibit 6: Calling both except & noexcept in try/catch block and § 4.7 Exhibit 7: Calling only except in try/catch block both show no difference introduced by labeling the functions as noexcept, due to the except poisoning mentioned earlier. You can see the disassembly of `noexcept_calls_mixed_in_try_catch` with the Itanium catch block APIs `__cxa_begin_catch` and `__cxa_end_catch`.

```
080015dc <noexcept_calls_mixed_in_try_catch()>:
80015dc: b508        push  {r3, lr}
80015de: f7ff fcc3   bl  8000f68 <bar()>
80015e2: e8bd 4008   ldmia.w sp!, {r3, lr}
80015e6: f7ff bc9b   b.w 8000f20 <noexcept_baz()>
80015ea: f000 f9b9   bl  8001960 <__cxa_begin_catch>
80015ee: 4a03        ldr r2, [pc, #12]
80015f0: 6bd3        ldr r3, [r2, #60] @ 0x3c
80015f2: 3301        adds  r3, #1
80015f4: 63d3        str r3, [r2, #60] @ 0x3c
80015f6: f000 f9f3   bl  80019e0 <__cxa_end_catch>
80015fa: bd08        pop {r3, pc}
80015fc: 20000b60  .word 0x20000b60
```

§ 6. Conclusion

Noexcept can be useful in cases such as where a strong exception guarantee is needed. But in terms of code gen, it's a mixed bag. In general, adding the `noexcept` to a function reduces its code gen.

The cases where GCC was able to optimize the code gen would be:

1. Removing destructor landing pads
2. Removing dead catch blocks

Removing destructor landing pads is useful, but making a function noexcept for this purpose seems a bit extreme.

Noexcept also tends to cause GCC to change what would have been 0 bytes of exception data into requiring a mostly empty LSDA section and exception index entry.

The benefits of noexcept only occur as an edge case. An edge case that breaks once a single function capable of throwing an exception is called within that function.

We want to give the compiler as much information and guarantees as possible to coax it into generating more efficient code for us, but do we really need `noexcept` for that?

Given the data in this study, I believe the right choice is to look for improvements in toolchains. Changes to code should not be necessary because there are many of the improvements to code gen that can be performed without the need to change source code.

§ 6.1. Improve data structure selection

The exception rank for noexcept functions could be optimized to choosen lower rank options. Here are a few checks that could be performed:

1. Is it a leaf? No entry
2. Calls only noexcept? No entry
3. Is noexcept and does not have a try block? inline noexcept

4. Is noexcept with non-trivial destructors? inline noexcept

§ 6.2. Group functions with identical exception entries

GCC merges identical exception entries when the functions are right next to each other in the source code. The linker could generate a first run of the exception index, collect all of the identical entries, and then group all functions with identical entries. Now all of the identical entires can be merged into a single entry, reducing the size of the table.

§ 6.3. Deduce `noexcept` in Functions

Many functions operate as `noexcept` without being explicitly marked as such; they call other functions, and down the entire call graph, no function ever throws an exception. Instead of manually marking such functions as `noexcept`, it is feasible for the linker to determine whether a function is exception propagating. GCC, for instance, can already generate a call graph using the `-fcallgraph-info` flag. The proposed idea is for the linker to evaluate all functions it has full assembly information of and determine which ones throw exceptions. Using this information, the linker could identify functions that could never throw an exception and automatically mark all them `noexcept`. Leaf functions would receive an implicit `noexcept` marking. Similarly, if an except functions calls a set of functions that have implicitly marked `noexcept` then that function could also be marked as implicitely noexcept. For APIs external to an application, such as those in a shared library, the linker would have to assume that any function not explicitly marked as `noexcept` does propagate exceptions.

Implementing such a mechanism would allow a C++ application to benefit from marking many of its non-throwing functions as `noexcept`, while retaining the flexibility to adjust this designation as needed in the future.